

Terminal Antifraude com TinyML

Cada compra decidida dentro do ESP32-S3, sem enviar os dados do cartão para a nuvem

Daniel Quiteque

Pós-graduação em IA Aplicada · UniSENAI · Prof. MSc. Rodrigo Kobashikawa Rosa

github.com/danielquiteque/projeto1-antifraude-tinym1

Hoje, a maquininha pergunta à nuvem se a compra é fraude

1

Privacidade

Dados do cartão e hábitos do cliente trafegam e ficam em terceiros (LGPD).

2

Sem rede, sem proteção

Metrô, zona rural, falha de operadora: ou recusa a venda, ou aprova às cegas.

3

Latência e custo

Cada consulta ao servidor custa banda, processamento e tempo de espera.

Aplicações de um antifraude embarcado

Maquininhas (POS)

Catracas de transporte

Vending machines e totens

Wearables e IoT

Regiões sem rede

As 4 etapas do projeto final, no mesmo dispositivo

Etapa exigida	Como foi atendida
1 · Coleta de sensores	Teclado 4×4 (valor, categoria) + RTC DS1307 via I2C (hora)
2 · Treinamento	MLP com dataset público de 1,78 milhão de transações de cartão
3 · Conversão e compressão	9 variantes comparadas → QAT int8 de 4.168 bytes
4 · Inferência no dispositivo	Sensores → 22 features → int8 → TFLite Micro → LED, buzzer, OLED, MQTT

Meta: qualidade igual à do modelo em ponto flutuante, em milissegundos, cabendo em poucos kilobytes do ESP32-S3.

Muitos dados, pouquíssima fraude

1,78 mi

transações (2019–2020)

0,53 %

são fraude (9.427)

993

cartões · 14 categorias

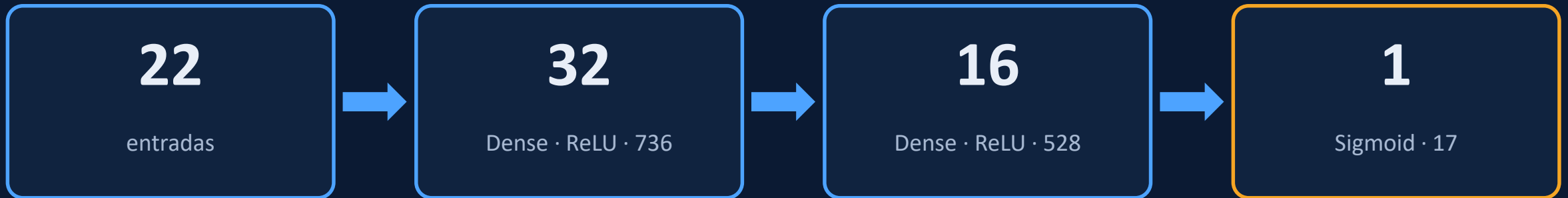
O que os dados mostram	Legítima	Fraude	Leitura
Valor médio	≈ US\$ 32	≈ US\$ 259	~8× maior
Hora (cosseno)	0,00	0,73	madrugada
Tempo desde a compra anterior	≈ 2,7 h	≈ 1,2 h	rajadas
Valor vs. média do cartão	−0,66	+0,36	fora do padrão

Do dado bruto às 22 entradas da rede

Variável bruta	Tratamento	Por quê
Valor (teclado)	$\log(1 + \text{valor})$	comprime a cauda longa
Hora (RTC)	seno e cosseno de $2\pi \cdot \text{hora}/24$	23h fica perto de 1h
Nascimento	idade em anos	perfil do titular
Timestamp	$\log(\Delta t \text{ desde a compra anterior})$; 1ª compra = 7 dias	detecta rajadas
Histórico	$\log(\text{valor}) - \log(\text{EMA } \alpha = 0,1)$; 1ª compra = 0	fora do padrão do cliente
Últimas 24 h	$\log(1 + n^\circ)$ e $\log(1 + \text{soma})$, sem a compra atual	velocidade de gasto
Categoria	one-hot (14 colunas de 0/1)	sem ordem artificial

As 8 contínuas são padronizadas, $(x - \mu) / \sigma$, com μ e σ só do treino (sem vazamento) e gravados no header C do firmware.

Um MLP de 1.281 parâmetros



Por que MLP

Entrada já é um vetor de 22 features;
usa só 2 operadores no TFLite Micro

Métrica

PR-AUC e recall: com 0,53 % de fraude
a acurácia engana

Limiar

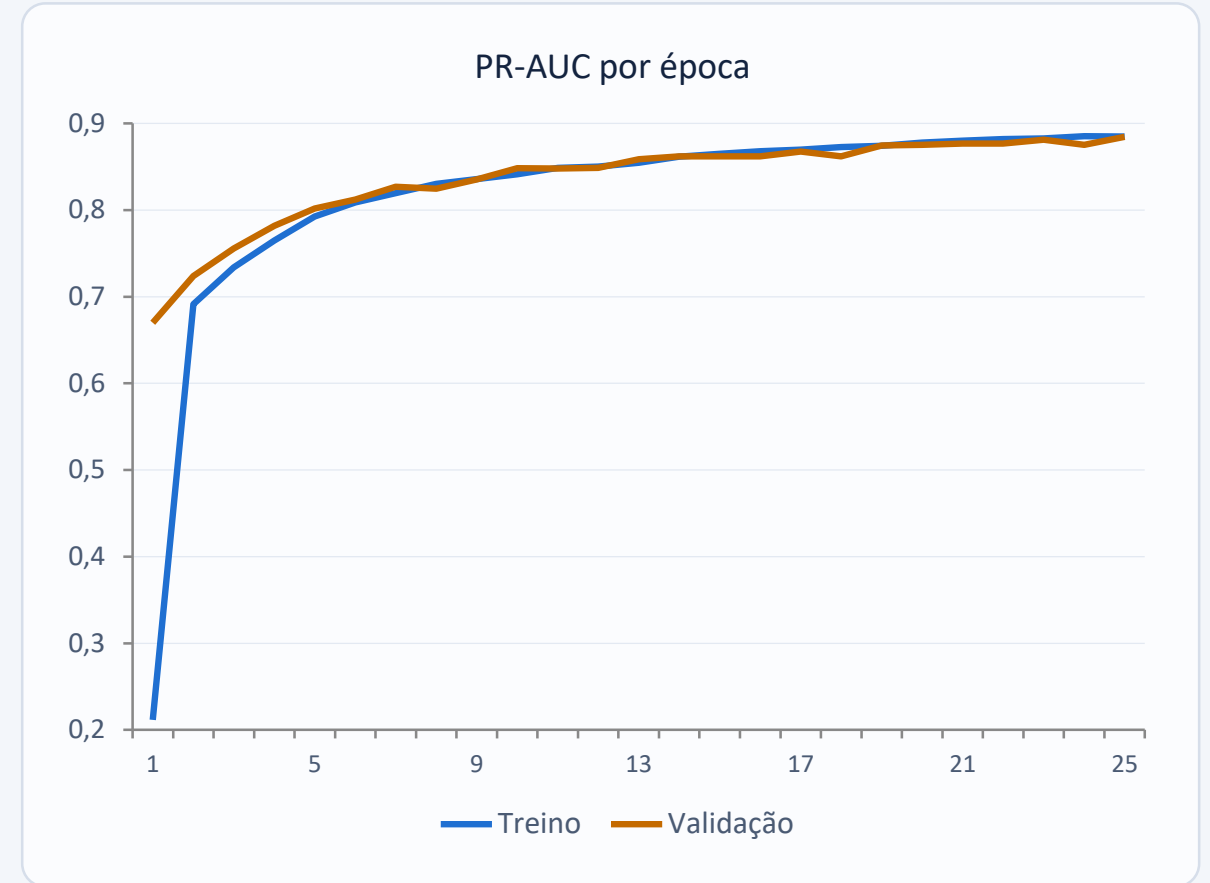
Recall ≥ 90 % na validação $\rightarrow P \geq 7,4$ %
($q \geq -109$ em int8)

25 épocas, sem decorar o cliente

Conjunto	Transações	Cartões
Treino 70 %	1.268.389	695
Validação 15 %	255.596	149
Teste 15 %	259.225	149

Hiperparâmetros: Adam (lr 0,002), entropia cruzada binária, batch 2.048 (620 passos/época), até 25 épocas, early stopping no PR-AUC de validação

Desbalanceamento: class_weight testado e descartado (PR-AUC pior); tratado no limiar, escolhido para recall ≥ 90 %



QAT int8: menor modelo, mesma qualidade

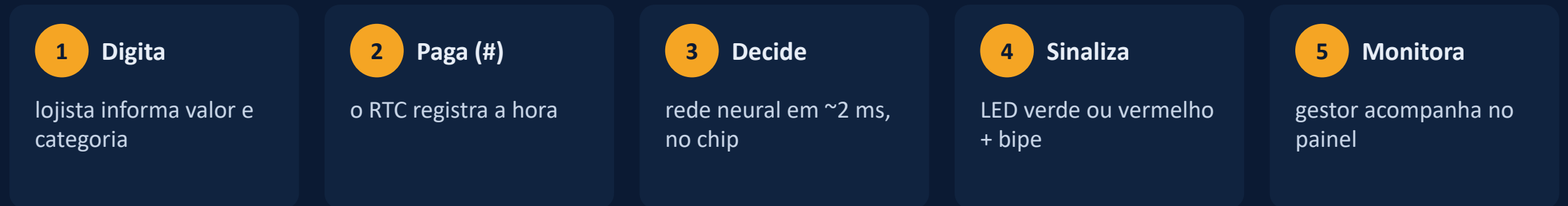
Modelo	.tflite	Arena	PR-AUC	Roda no TFLM?
float32 (base)	7.156 B	1.696 B	0,899	sim
float16 / dynamic range	5.388 B	—	0,899	não
PTQ int8	4.968 B	1.984 B	0,894	sim
QAT int8 · embarcado	4.168 B	1.600 B	0,899	sim · escolhido
pruning 75 % + int8	4.968 B	1.984 B	0,739	sim
clustering 16 + int8	4.968 B	1.984 B	0,776	sim
aluno destilado 16-8	3.536 B	1.776 B	0,784	sim

Por quê: fp16 não roda no microcontrolador · PTQ perde 0,005 · QAT recupera e encolhe · pruning e clustering só ganham no gzip e custam PR-AUC.

A decisão é local; a nuvem só recebe o resultado

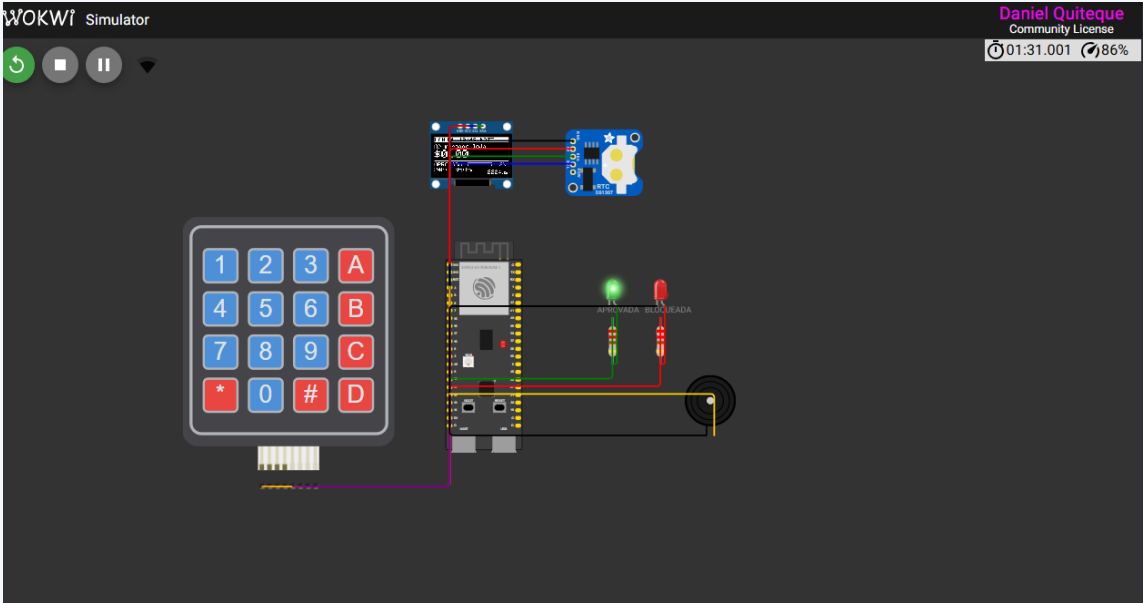


Jornada de uso no caixa



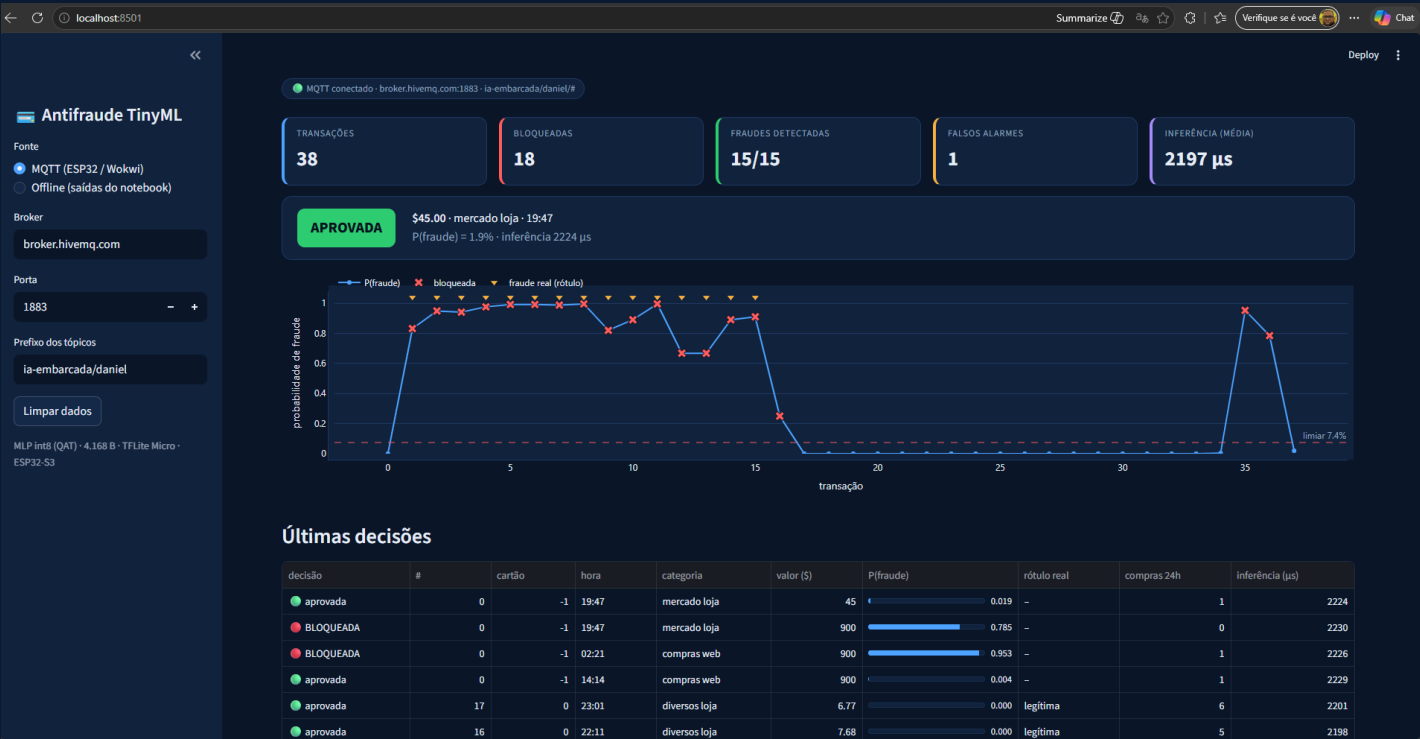
O terminal simulado no ESP32-S3

Componente	Função	Pinos
ESP32-S3-DevKitC-1	MCU · 512 KB SRAM	—
Teclado 4×4	valor, categoria	GPIO 4–7, 15–18
RTC DS1307	hora da compra	I2C 8/9 · 0x68
OLED SSD1306	tela	I2C 8/9 · 0x3C
LED verde + 220 Ω	APROVADA	GPIO 10
LED vermelho + 220 Ω	BLOQUEADA	GPIO 11
Buzzer	bipe (PWM)	GPIO 12



Firmware ESP-IDF 5.4.2: main.cc · fraud_features.c · fraud_model.cc (TFLite Micro, 2 operadores, arena estática) · drivers

Cada decisão chega ao painel ao vivo



Como o dado viaja

- 1 • **Envio:** ESP32 publica 1 JSON por compra via Wi-Fi, MQTT QoS 0
- 2 • **Broker:** HiveMQ repassa, não guarda
- 3 • **Recebe:** painel assina o tópico numa thread (paho-mqtt)
- 4 • **Armazena:** últimas 2.000 decisões em memória
- 5 • **Exibe:** tela atualiza a cada 1 s

Mesma compra, outra hora, outra decisão

#	Compra (teclas)	Hora	P(fraude)	Decisão
1	US\$ 45 · mercado loja	atual	≈ 0 %	APROVADA (verde)
2	US\$ 900 · compras web	14:14	0,4 %	APROVADA (verde)
3	US\$ 900 · compras web (B até 02h)	02:21	95,3 %	BLOQUEADA (vermelho)
4	Replay de dados reais (tecla D)	—	—	15/15 fraudes da rajada

A tecla B adianta o RTC: o modelo usa a hora, o histórico do cartão e o intervalo entre compras, não só o valor.

Previsto pelo simulador: q = 117. Medido no ESP32: **q = 116**. Comportamento reprodutível.

A qualidade do float32 em 4 KB e 2 ms

Matriz de confusão no ESP32 · replay de 133 compras

	previu legítima	previu fraude
real legítima	87 aprovadas (VN)	1 falso alarme (FP)
real fraude	1 fraude perdida (FN)	44 bloqueadas (VP)

Teste completo (259 mil): precisão 0,635 · recall 0,892 · F1 0,742

4.168 B

modelo QAT int8

812 B

arena no ESP32

~2,2 ms

por inferência

0,899

PR-AUC = float32

Um chip barato decide sozinho, com qualidade de float32

Entregue

- Pipeline completo do dado ao dispositivo
- Features calculadas no chip a partir de sensores
- 9 técnicas de compressão comparadas com dados
- Decisão local + telemetria MQTT + dashboard

Limitações e próximos passos

- Dados sintéticos e drift temporal → re-treino e OTA
- Hardware real com ESP-NN e medição de energia
- Leitor NFC e estado dos cartões na Flash
- MQTT com TLS e broker privado

Obrigado! Perguntas?